HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

CTF CHALLENGE WRITE-UP

Web Exploitation

BKSEC: Gambling Coin 1

DucTapCodeDao 202416788

Mục lục

1 Phân tích Challenge **3**

2 Test thử 1 lượt chơi để xem Request và Response của trang web **3**

3 Chèn thử 1 vài payload và rút ra kết luận **5**

4 Viết code Python để tìm flag **5**

1 Phân tích Challenge

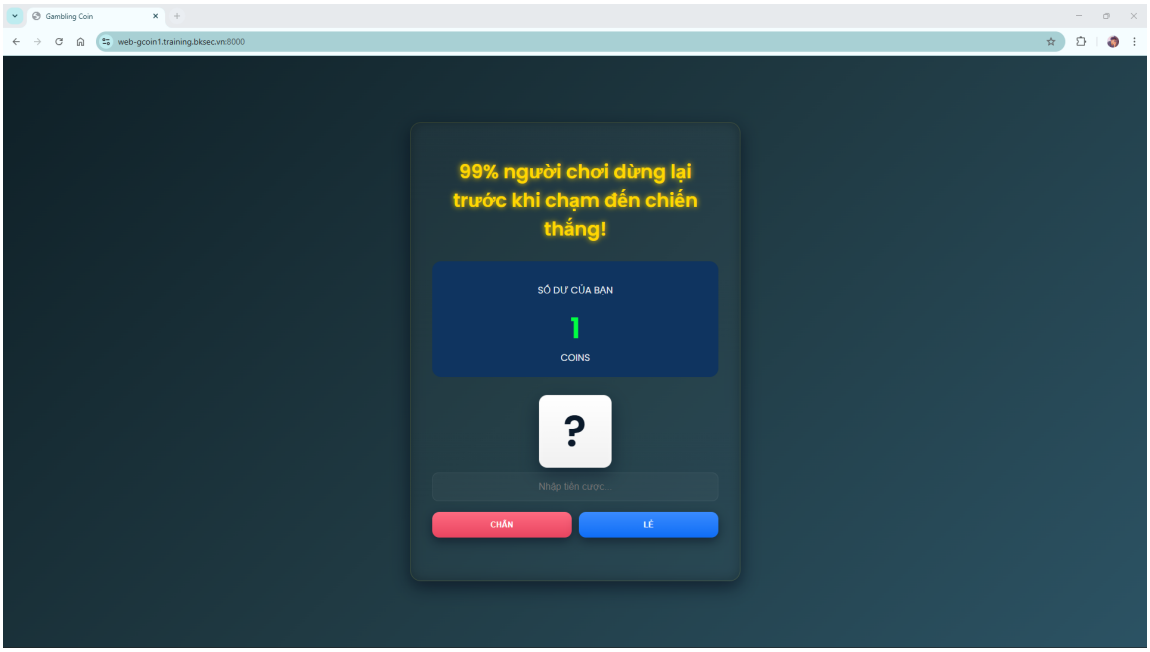
Challenge cho chúng ta một đường link:

```
https://web-gcoin1.training.bksec.vn:8000/
```

cùng với một đoạn gợi ý:

> *"A simple even/odd betting game. You start with 1 coin and can bet on CHẴN (even) or LẺ (odd). Your goal is to find a way to reach 999 coins to retrieve the flag.*
>
> *Disclaimer: This challenge does not encourage or promote gambling in any form.*
>
> *Khuyến cáo: Thử thách này được tạo ra hoàn toàn vì mục đích giáo dục và giải trí. CLB BKSEC không ủng hộ, quảng bá hay khuyến khích bất kỳ hành vi đánh bạc nào."*

Duc nhận định challenge này yêu cầu người chơi phải tìm cách nào đó để số **coins** vượt qua con số **999**, khi này flag sẽ xuất hiện. Trước tiên, **Duc** mở trang web bằng trình duyệt *Google Chrome*, và đây là giao diện của trang web:



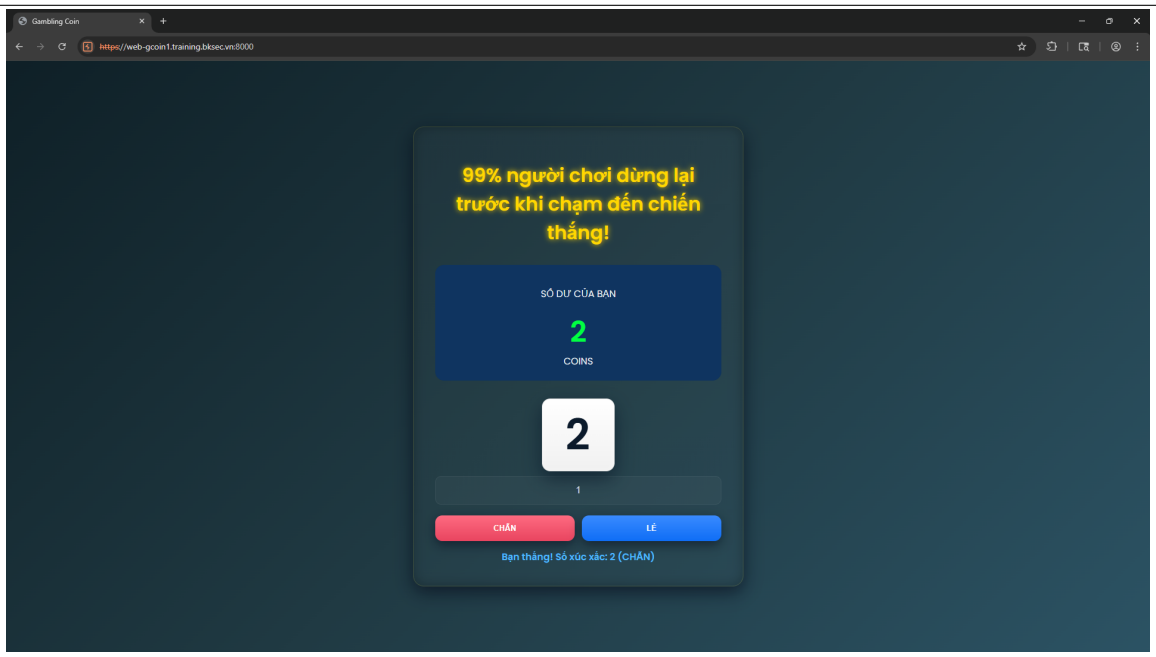
Hình 1: Giao diện khi mở đường link ở trình duyệt thông thường (Google Chrome).

Nhìn vào giao diện, Duc nhận thấy đây là một mini game **Gambling Coin**. Ở giữa màn hình là một khung trò chơi hiển thị slogan *"99% người chơi dừng lại trước khi chạm đến chiến thắng!"*, kèm theo phần **Số dư của bạn** (hiện tại là **1 coin**). Bên dưới là một ô nhập *Nhập tiền cược...*, và hai nút lựa chọn **CHẴN** và **LẺ** để đặt cược theo kết quả *even/odd*.

Để xác định chính xác hướng tấn công, Duc ném link vào *Burp Suite* để bắt đầu vọc vạch và phân tích request/response.

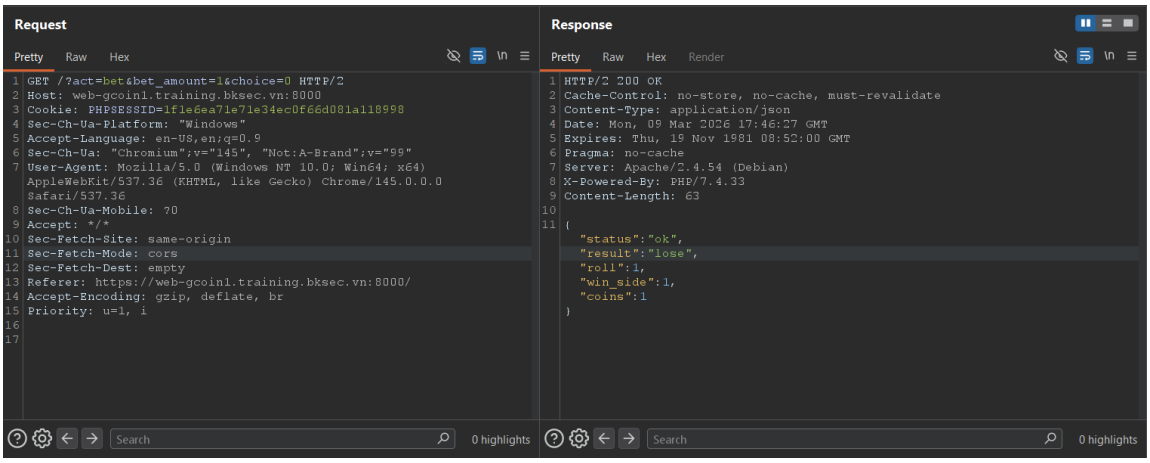
2 Test thử 1 lượt chơi để xem Request và Response của trang web

Để kiểm tra cách trang web phản hồi, Duc quyết định **chơi thử 1 lượt**. Duc cược **1 coin** và chọn **CHẴN**. Trang web phản hồi như sau:



Hình 2: Kết quả sau khi cược 1 coin và chọn CHẴN.

Tiếp theo, Duc mở *Burp Suite* để đọc **request/response** của hành động vừa thực hiện:



Hình 3: Request và response tương ứng khi thực hiện một lượt cược trong game.

Theo như quan sát thì có vẻ response chỉ trả về những thông tin đúng như trên giao diện, và không cung cấp thêm manh mối nào thật sự "đắt giá".

Chuyển sang phía request, dòng đầu tiên:

```
GET /?act=bet&bet_amount=1&choice=1 HTTP/2
```

gợi ý rằng ta có thể **chủ động chỉnh tham số** `bet_amount` (số tiền cược) trực tiếp trên request, thay vì chỉ phụ thuộc vào thao tác trên giao diện.

Để có thêm manh mối, Duc nhấn `F12`, mở tab *Sources* và đọc file `game.js`. File này cho Duc thêm **2 thông tin quan trọng**:

- **Thuật toán random nằm ở backend:** Cơ chế random ra kết quả được xử lý phía server (cụ thể là trong `index.php`), và ta không thể truy cập/đọc trực tiếp file này. Vì vậy, hướng tác động vào thuật toán random gần như là **không khả thi**.

- **Client-side validation cho số tiền cược:** Trong `game.js` có một mệnh đề `if` chỉ cho phép số tiền cược **lớn hơn 0**. Nếu không thoả mãn, trang web sẽ báo lỗi "Vui lòng nhập số tiền cược hợp lệ".

```
if (!bet || bet <= 0) {
        msg.innerText = 'Vui lòng nhập số tiền cược hợp lệ';
        return;
    }
```

Hình 4: Mệnh đề `if` trong `game.js` chỉ cho phép `bet > 0`.

Điều này làm phá sản ý tưởng ban đầu của Duc là chèn số tiền cược kiểu `-998`. Vì vậy, thay vì dùng số âm, Duc phải chuyển sang tìm cách chèn **một kiểu giá trị khác**.

3 Chèn thử 1 vài payload và rút ra kết luận

Để tiện cho việc thay đổi số tiền cược, Duc bật *Intercept* trong Burp Suite, sau đó thử với payload đầu tiên là `0.75`. Kết quả trả về cho thấy trang web vẫn chạy bình thường và số tiền cược thực tế được ghi nhận là **1**.

Điều này chứng minh rằng `0.75` đã vượt qua lớp validate đầu tiên (`bet > 0`) ở phía client, nhưng đã bị một hàm xử lý ở backend (trong `index.php`) **làm tròn** hoặc **ép kiểu** thành số nguyên `1`.

Tiếp theo, Duc thử payload `999e-3` (dạng ký hiệu khoa học). Mục đích của payload này là:

- `999e-3 = 0.999 > 0` nên chắc chắn qua được validate `bet > 0`.
- Nếu backend xử lý input kém (không parse dạng scientific notation đúng cách), nó có thể **hiểu nhầm** chuỗi `999e-3` thành `999`.

Tuy nhiên, trang web vẫn hoạt động bình thường và tiếp tục ghi nhận `bet = 1`.

Với việc backend xử lý input quá chặt (ép kiểu/làm tròn/parse số rất nghiêm ngặt), Duc rút ra kết luận:

> **Kết luận:** Bài này gần như không thể "bẻ" bằng cách nhồi payload vào `bet_amount`. Thay vào đó, hướng đi hợp lý hơn là **lợi dụng logic của trang web** để thỏa mãn điều kiện (đạt `999 coins`), từ đó lấy flag.

4 Viết code Python để tìm flag

Duc nhận ra nếu luôn **tất tay (all-in)** sau mỗi lần cược thì số tiền sẽ tăng theo **cấp số nhân**:

$$1 \to 2 \to 4 \to 8 \to 16 \to 32 \to 64 \to 128 \to 256 \to 512 \to 1024$$

Vì $1024 > 999$ nên sẽ thỏa mãn điều kiện để lấy **flag**. Tỉ lệ để chuỗi thắng liên tục 10 ván xảy ra là:

$$0.5^{10} = \frac{1}{1024} \approx 0.0977\% \approx 0.1\%.$$

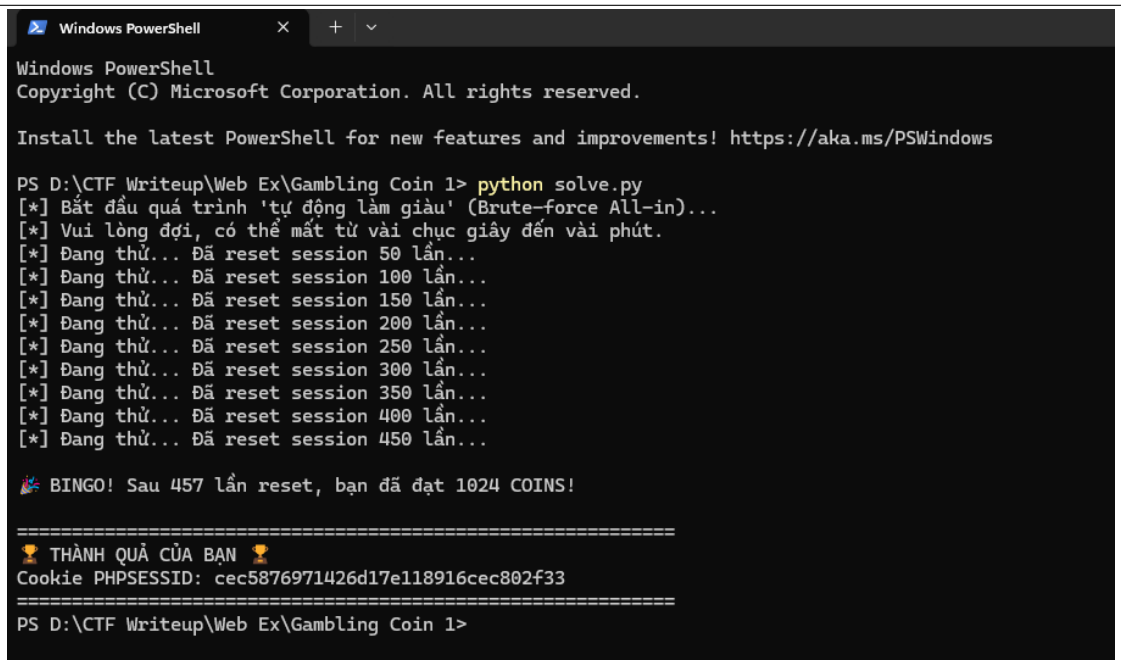
Nếu làm bằng tay thì gần như không thể, nhưng nếu viết code để chạy lặp thì hoàn toàn có thể xảy ra.

Vì vậy nên Duc nhờ **Gemini** viết một đoạn code Python có cơ chế: *mỗi lần cược luôn all-in và chọn bên chẵn; nếu thắng thì tiếp tục; nếu thua thì tạo session mới và cược lại từ đầu; lặp cho đến khi số tiền đạt 1024 thì dừng lại.* Và đây là **hàm xử lý cơ chế** này:

```python
coins = 1

    while 0 < coins < 999:
        params = {
            "act": "bet",
            "bet_amount": coins,
            "choice": 0 # Liên tục All-in vào CHẴN
        }

        try:
            res = session.get(url, params=params, verify=False, timeout=5)
            data = res.json()

            if data.get("result") == "win":
                coins = data.get("coins") # Thắng thì update số tiền để All-in tiếp
            else:
                break # Thua thì gãy chuỗi, thoát vòng lặp nhỏ để tạo session mới
```

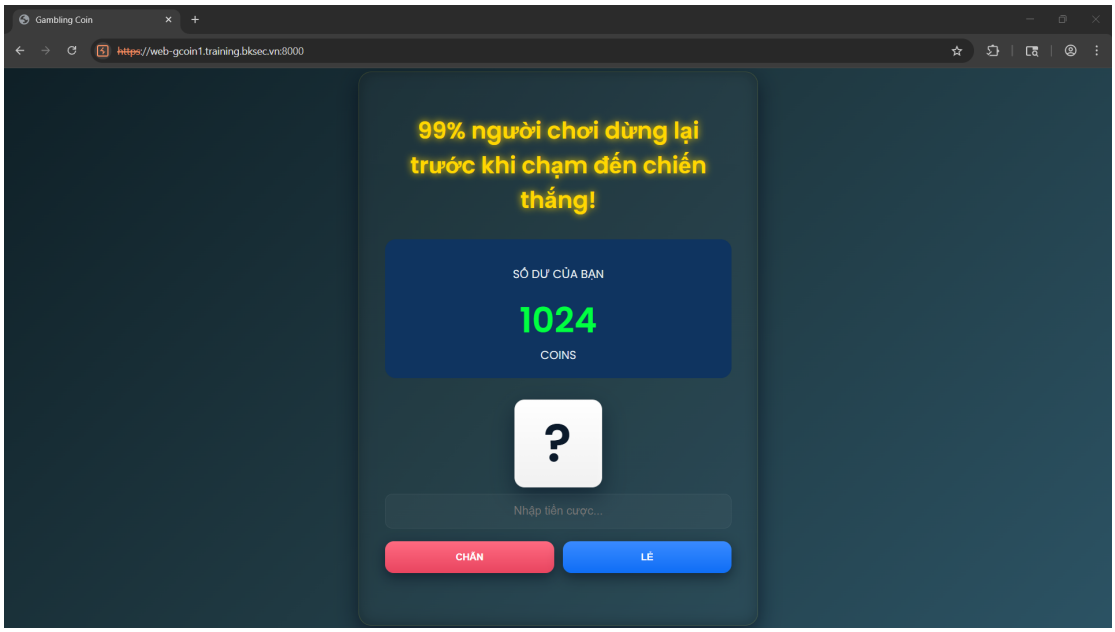
Hình 5: Hàm Python thực hiện chiến thuật all-in liên tục, thua thì reset session và thử lại.

Duc chạy thử script và sau **457 lần reset** thì đã thu được kết quả:



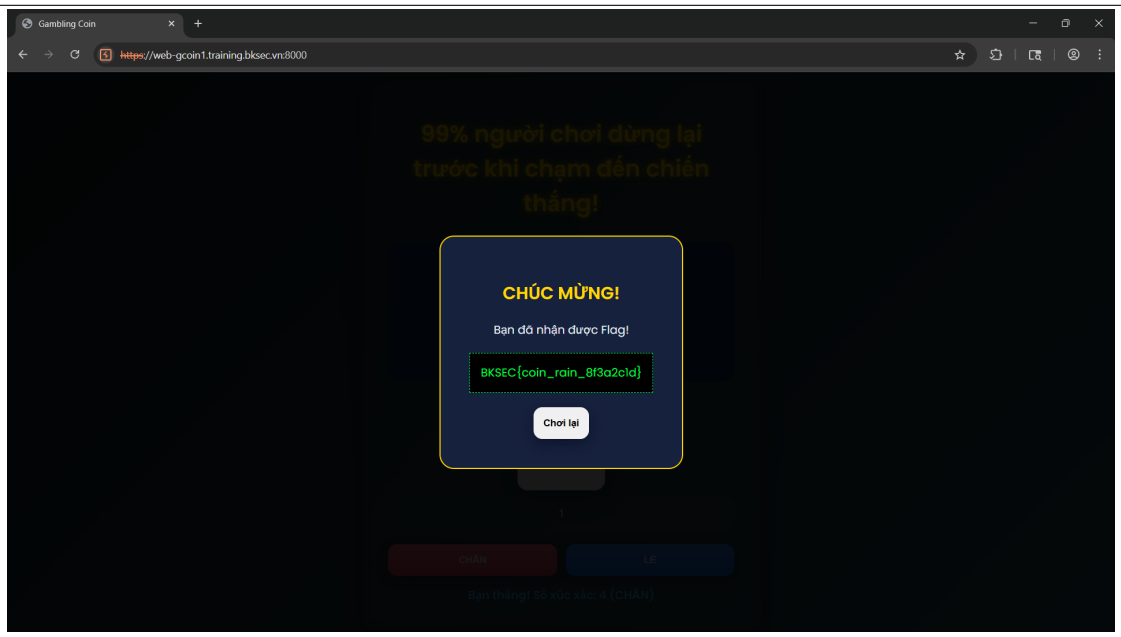
Hình 6: Output khi chạy script: đạt 1024 coins và thu được `PHPSESSID`.

Duc copy `PHPSESSID` rồi vào trang web, nhấn **F12** → tab **Application** → **Cookies**, thay đổi giá trị cookie tương ứng, sau đó **F5** để reload lại trang web. Trang web đã được update như sau:



Hình 7: Sau khi thay `PHPSESSID` và reload: số dư đã được cập nhật lên 1024 coins.

Giờ chỉ cần chơi thêm 1 ván nữa là số tiền auto > 999 và lấy được flag. Và đây là giao diện hiển thị flag:



Hình 8: Giao diện nhận flag sau khi số coins vượt 999.

Duc nhập thử flag được in ra, và cuối cùng xác định được **flag đúng** của bài này là:

`BKSEC{coin_rain_8f3a2c1d}`

Problem solved!